

Working with Existing Resources



Ned Bellavance

HashiCorp Ambassador

@ned1313 | nedinthecloud.com



Module Overview



Greenfield is the exception

Import options with Terraform

- Import command
- Import block



Globomantics Environment



GLOBOMANTICS

Using AWS for cloud provider

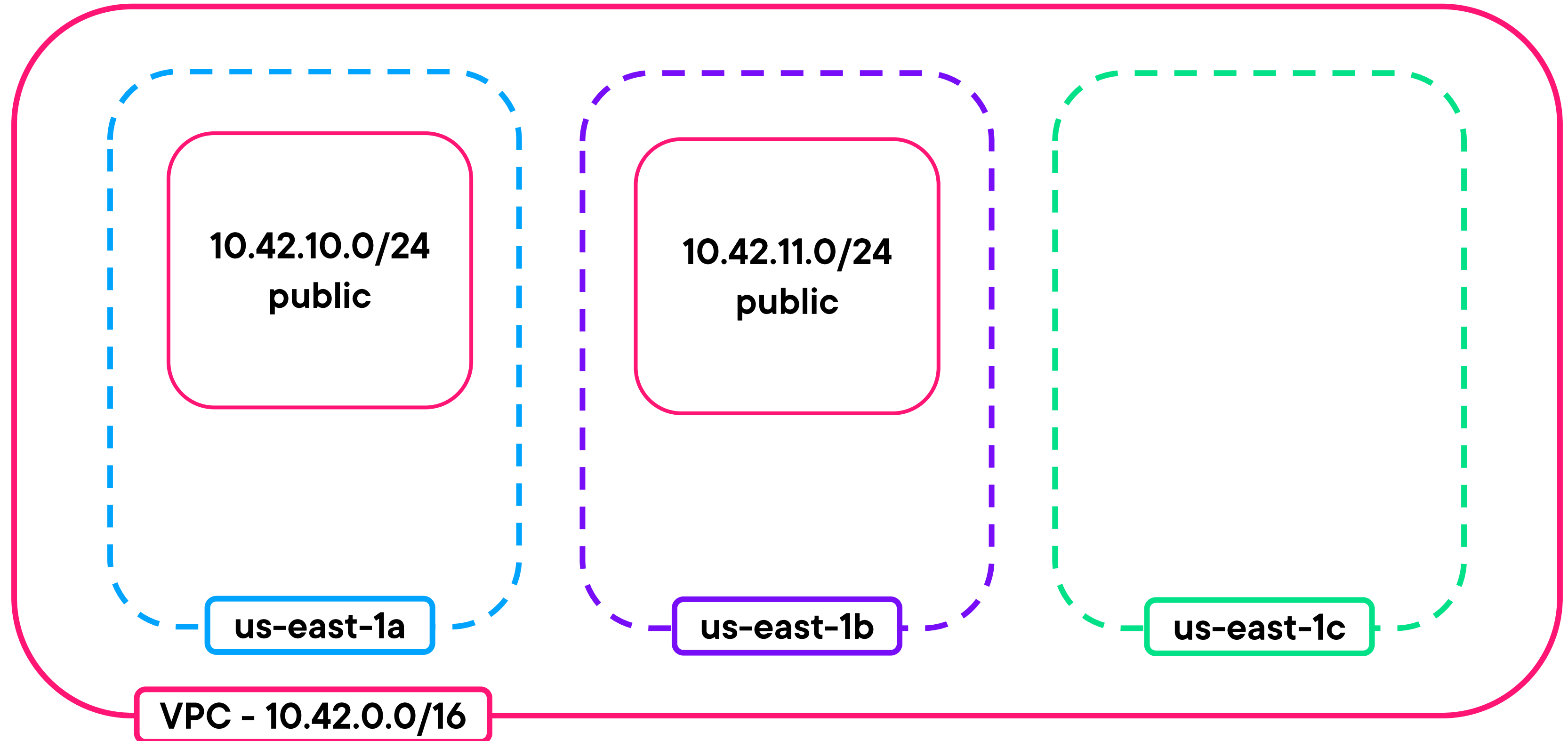
Terraform for all network deployments

Consistency and security

Some preexisting environments



Existing Environment





Examine the CloudFormation template

Deploy the initial VPC

Play along!

- AWS account
- AWS CLI
- Terraform CLI
- Exercise files



**Some of the resources
deployed in AWS may cost
money. Act accordingly.**





Importing Existing Resources



The Import Command

Command syntax

```
terraform import [options] ADDR ID
```

ADDR – configuration resource identifier

Ex. – `aws_vpc.main`

ID – provider specific resource identifier

Ex. – `vpc-123654789`

Importing a vpc into a configuration

```
terraform import aws_vpc.main vpc-123654789
```



Import Command Process

```
terraform import aws_vpc.main vpc-123654789
```

terraform.tfstate

```
{
  "resources": [
    {
      "mode": "managed",
      "type": "aws_vpc",
      "name": "main",
      "instances": [
        {
          "id": "vpc-123654789"
        }
      ]
    }
  ]
}
```

main.tf

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support = true

  tags = {
    Name = var.vpc_name
  }
}
```



Import Command Process

main.tf

```
resource "aws_vpc" "main" {  
  cidr_block      = var.vpc_cidr  
  enable_dns_support = true  
  
  tags = {  
    Name = var.vpc_name  
  }  
}
```

```
> terraform plan
```

Waiting for the plan to
start...

Terraform v1.5.0

...

No changes. Your
infrastructure matches the
configuration.



imports.tf

Import Blocks

```
import {  
  to = address  
  id = identifier  
}
```



Import Blocks

imports.tf

```
import {  
  to = "aws_vpc.main"  
  id = "vpc-123654789"  
}
```

```
import {  
  to = "aws_subnet.subnet1"  
  id = "subnet-123654789"  
}
```

```
import {  
  to = "aws_subnet.subnet2"  
  id = "subnet-789654123"  
}
```



Import Command Process

```
terraform plan -generate-config-out=generated.tf
```

imports.tf

```
import {  
  to = "aws_vpc.main"  
  id = "vpc-123654789"  
}
```

generated.tf

```
# __generated__ by Terraform from...  
  
resource "aws_vpc" "main" {  
  cidr_block          = "10.42.0.0/16"  
  enable_dns_support = true  
  
  tags = {  
    Name = "globo-web-app"  
  }  
}
```



Import Command Process

Plan the imports

main.tf

```
import {  
  to = "aws_vpc.main"  
  id = "vpc-123654789"  
}  
  
resource "aws_vpc" "main" {  
  cidr_block      = var.vpc_cidr  
  enable_dns_support = true  
  
  tags = {  
    Name = var.vpc_name  
  }  
}
```

> terraform plan

Terraform will perform the following actions:

...

Plan: 1 to import, 0 to add,
0 to change, 0 to destroy.



Import Command Process

Apply the imports

main.tf

```
import {  
  to = "aws_vpc.main"  
  id = "vpc-123654789"  
}  
  
resource "aws_vpc" "main" {  
  cidr_block      = var.vpc_cidr  
  enable_dns_support = true  
  
  tags = {  
    Name = var.vpc_name  
  }  
}
```

```
> terraform apply
```

```
...
```

Apply complete! Resources:
1 imported, 0 added,
0 changed, 0 destroyed.



Choosing an Import Process

Import command	vs	Import block
- Directly edits state data - Single resource at a time - Create configuration manually - Supported at least through v1.X		- Produces execution plan - Multiple resources - Terraform generated blocks - Preferred method going forward



Current Gaps with Import

No dynamic discovery of existing infrastructure

Generated resource configuration blocks are literal



Module Summary



Brownfield is the norm

Import small chunks at a time

Generated config is a starting point



Up Next:

Enabling Team Collaboration

